The Capacitated Vehicle Routing Problem (CVRP)

CSE 462: Algorithm Engineering Sessional
Final Presentation

Team Members

Sabbir Alam Saad	(ID: 2005059)
Tanvir Hossain	(ID: 2005014)
Md. Afzal Hossain Tanim	(ID: 2005008)
Reduanul Islam Imon	(ID: 2005040)
Suhaeb Bin Matin	(ID: 2005086)
Mohammad Ali Bhuiyan	(ID: 2005113)

Department of Computer Science and Engineering (CSE)
Bangladesh University of Engineering and Technology (BUET)

30th March 2026

What is the Capacitated Vehicle Routing Problem?

Definition

The Capacitated Vehicle Routing Problem (CVRP) is a combinatorial optimization problem where a fleet of vehicles with limited capacity must deliver goods to multiple customers from a central depot, minimizing total travel distance or cost [1].

Central Depot	Fleet of Vehicles	Limited Capacity	Customer Demands
Starting and ending point for all vehicles	Multiple vehicles used to serve all customers	Each vehicle has maximum load capacity	Each customer has specific delivery requirements

Key Components of CVRP

- **Depot (Central Warehouse):** A single starting location where all vehicles begin and end their routes
- **Vehicle Fleet:** A set of m identical or homogeneous vehicles, each with capacity Q
- **Customers:** A set of n customers, each located at specific coordinates with demand d_i
- **Distance Matrix:** Known distances or costs c_{ij} between all pairs of locations (depot and customers)
- **Capacity Constraint:** Total demand on any route cannot exceed vehicle capacity Q

CVRP Mathematical Formulation

Objective Function

$$\text{Minimize: } \sum_i \sum_j c_{ij} \cdot x_{ij}$$

(Total travel cost / distance)

Constraints

1. **Visit each customer once:**

$$\sum_j x_{ij} = 1 \quad \forall i$$

2. **Capacity constraint:**

$$\sum_i d_i \leq Q$$

3. **Flow conservation:**
   Flow in = Flow out
4. **Depot condition:**
   Start and end at depot

CVRP Example

Problem Setup:

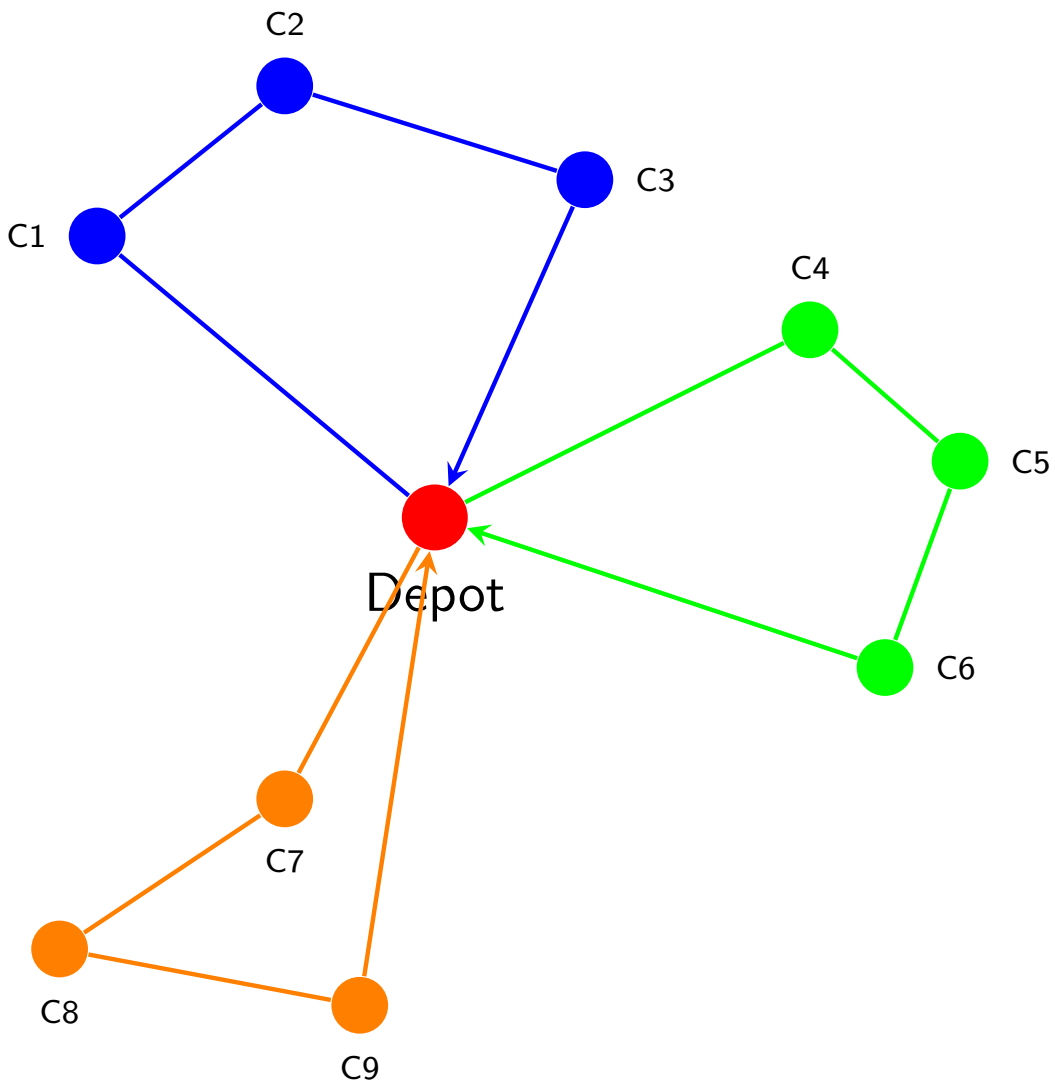
- Depot: Central warehouse
- Vehicles: 3 trucks
- Capacity: 100 units each
- Customers: 9 locations

Customer Demands:

C1: 30	C2: 25	C3: 40
C4: 35	C5: 45	C6: 20
C7: 50	C8: 30	C9: 15

Solution:

- Route 1: 95 units
- Route 2: 100 units
- Route 3: 95 units



Proving CVRP is NP-Hard

Our Approach

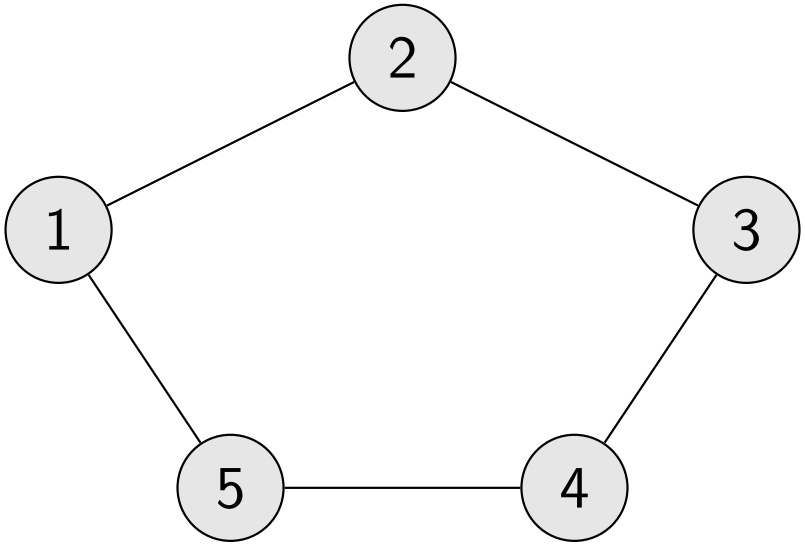
We prove CVRP is NP-Hard by showing a polynomial-time reduction from the Traveling Salesman Problem (TSP).

Known:
TSP is NP-Hard

To Prove:
CVRP is NP-Hard

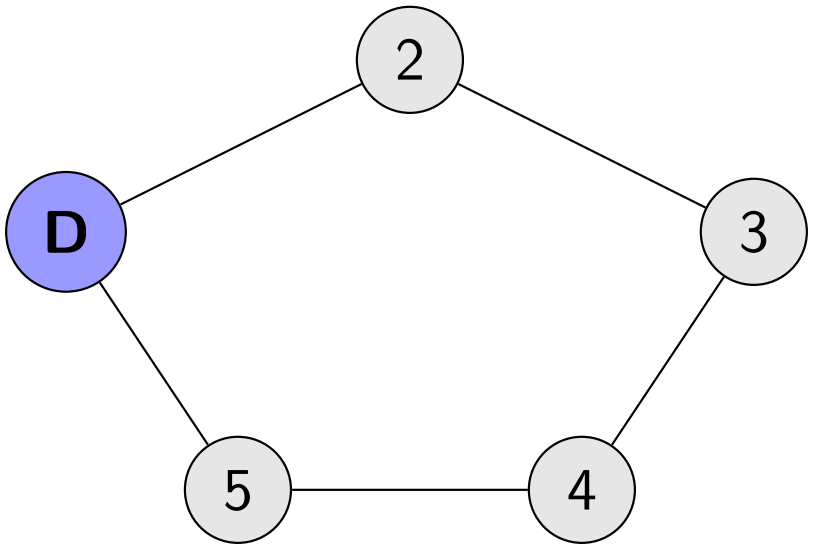
$$\text{TSP} \leq_P \text{CVRP}$$

Step 1: Traveling Salesman Problem (TSP)



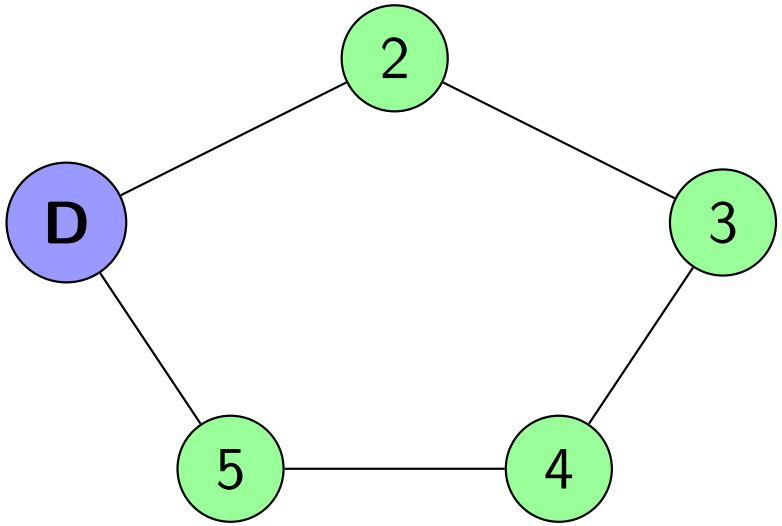
All nodes are identical — find a minimum Hamiltonian cycle

Step 2: Choose a Depot



Select any node as the depot

Building the CVRP Instance (from TSP)

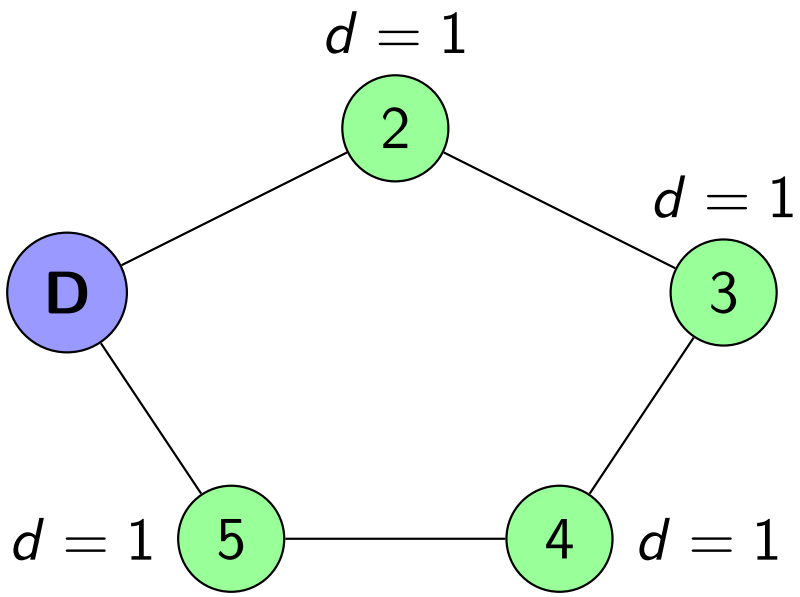


Define customers

CVRP Parameters

Customers: n - 1

Building the CVRP Instance (from TSP)

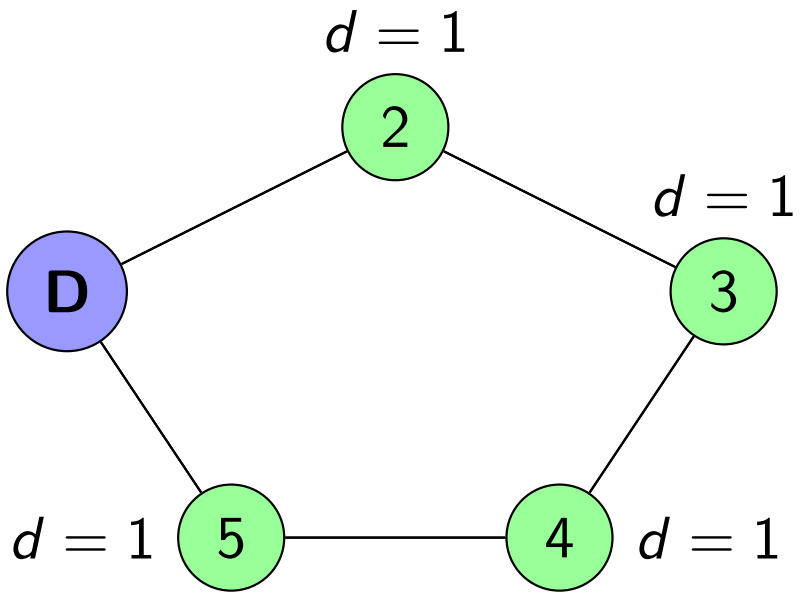


Assign unit demands

CVRP Parameters

Customers: n - 1
Demand: $d_i = 1$

Building the CVRP Instance (from TSP)

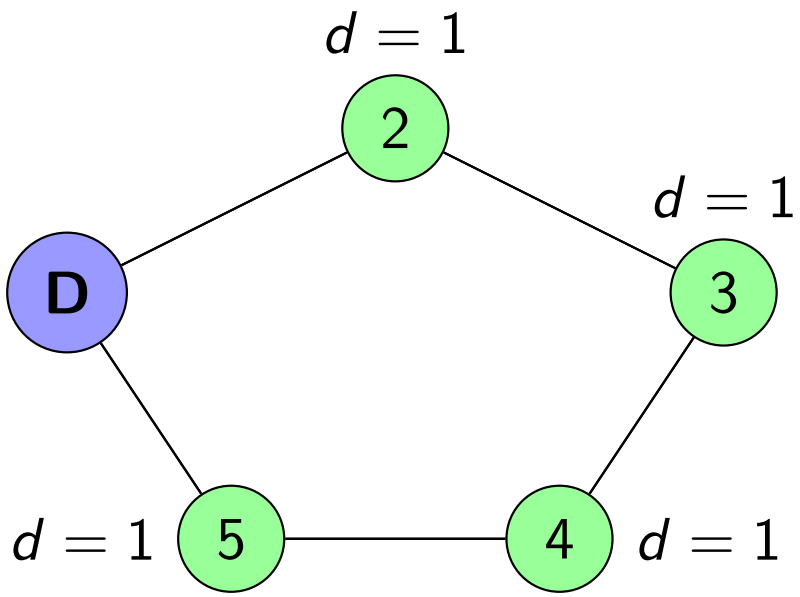


Restrict to one vehicle

CVRP Parameters

Customers: n - 1
Demand: $d_i = 1$
Vehicles: $m = 1$

Building the CVRP Instance (from TSP)



Capacity forces a single tour

CVRP Parameters

Customers: n - 1
Demand: $d_i = 1$
Vehicles: $m = 1$
Capacity: $Q = n - 1$

Key Insight

Crucial Observation

By setting

$$vehicles = 1, \quad demand = 1, \quad capacity = n - 1$$

Equivalence

One CVRP route visiting all customers $\Longleftrightarrow$ One TSP Hamiltonian cycle

NP-Hardness Conclusion

$$\text{TSP} \leq_p \text{CVRP}$$

Conclusion

Since TSP is NP-Hard, the Capacitated Vehicle Routing Problem is NP-Hard.

Algorithm Categories Defined I

Before detailing specific CVRP algorithms, we must define the classes of algorithms used in operations research.

1. **Exact Algorithms**
   - **Definition:** Algorithms that guarantee finding the *globally optimal* solution.
   - **Pros:** Proof of optimality.
   - **Cons:** Exponential time complexity ($O(2^n)$ or worse). Only feasible for small instances ($n < 200$).

Algorithm Categories Defined II

2. **Approximation Algorithms**
   - **Definition:** Polynomial-time algorithms that provide a guarantee on the solution quality.
   - **The Guarantee (α):** An algorithm is an α-approximation if for any instance, the solution $Sol \leq \alpha \times Opt$.
   - **Context:** For general CVRP, finding a constant factor approximation is difficult due to the hardness of Bin Packing.

Algorithm Categories Defined III

3. **Heuristics**
   - **Definition:** "Rule of thumb" techniques. They use problem-specific knowledge to find "good enough" solutions quickly.
   - **No Guarantees:** Unlike approximation algorithms, they do not guarantee a bound on the error.
   - **Types:**
     - *Constructive:* Build a solution from scratch.
     - *Improvement:* Take an existing solution and try to make it better.

Algorithm Categories Defined IV

4. Metaheuristics

- **Definition:** High-level problem-independent frameworks that guide local search heuristics to escape local optima.
- **Mechanism:** They balance *Exploration* (searching new areas of the solution space) and *Exploitation* (refining the current best solution).
- **Examples:** Genetic Algorithms, Tabu Search, Simulated Annealing.

Existing Algorithms

Category	Algorithm	Key Features
Exact	Branch and Bound [2]	Enumerates candidate solutions by forming a rooted tree
	Branch-and-Cut [1]	Uses capacity cuts and comb inequalities.
	Branch-Cut-and-Price [3]	State-of-the-art; uses Column Generation.
Approximation	Christofides' Ext. [4]	Extensions of TSP approx; mostly theoretical.
Heuristic	Clarke & Wright [5]	Greedy merging based on "savings" (s_{ij}).
	Sweep Algorithm [6]	Clusters by polar angle, solves TSP per cluster.
Metaheuristic	Tabu Search [7]	Local search with memory structures.
	Genetic Algorithm [8]	Population-based; often uses "Giant Tour" split.
	Large Neighborhood Search [9]	Destroy part of the solution and recreate it

Branch and Bound (B&B)

Concept: Systematically enumerates candidate solutions by forming a rooted tree [2].

Mechanism:

1. **Branching:** Splits the problem into smaller subproblems (e.g., "Edge x_{12} must be used" vs "Edge x_{12} cannot be used").
2. **Bounding:** Calculates a lower bound for each subproblem (usually via Linear Relaxation).
3. **Pruning:** If a subproblem's Lower Bound $>$ Current Best Solution, that branch is discarded (pruned).

Branch and Cut (B&C)

Enhancement to B&B: Standard B&B is weak because the Linear Relaxation of CVRP is often too loose. B&C adds *Valid Inequalities* (Cuts) to tighten the relaxation [1].

Famous Cuts:

- **Rounded Capacity Cuts:** Ensures demand constraints are met for subsets.
- **Comb Inequalities:** Generalized from TSP to handle connectivity.
- **Framed Capacity Cuts:** Specific to VRP, combining capacity and geometric constraints.

Status: Currently effective for $n \approx 100 - 150$.

Branch-Cut-and-Price (BCP)

The State-of-the-Art: Combines Branch-and-Bound with **Column Generation** [3].
Set Partitioning Formulation: Instead of deciding edges (x_{ij}), we decide which *Routes* (r_k) to select.

$$\min \sum_{r \in \Omega} c_r \lambda_r$$

Subject to: Each customer is in exactly one route.
Column Generation: Since the set of possible routes Ω is exponential, we start with a few. We generate new "promising" routes dynamically by solving a **Pricing Subproblem** (usually a Shortest Path Problem with Resource Constraints - SPPRC).
Status: Can solve instances up to $n \approx 200 - 300$.

Constructive Heuristics

These algorithms build a feasible solution from scratch.

A. Clarke and Wright Savings Algorithm [5]

- **Init:** n routes $(0 \to i \to 0)$.
- **Metric:** Savings $s_{ij} = c_{i0} + c_{0j} - c_{ij}$.
- **Step:** Merge routes with max s_{ij} if capacity allows.
- **Complexity:** $O(n^2 \log n)$.

B. Sweep Algorithm [6]

- **Geometric approach.**
- Sort customers by polar angle around the depot.
- "Sweep" around, adding customers to a vehicle until capacity Q is full, then start a new vehicle.
- Solve TSP for each cluster.

Tabu Search (TS)

Mechanism: [7]

- Performs local search (like Swap/Relocate).
- **The Tabu List:** To prevent cycling ($A \to B \to A$), recent moves are declared "Tabu" (forbidden) for a certain number of iterations.
- **Aspiration Criterion:** Can override Tabu status if the move produces a new global best solution.

Genetic Algorithms (GA)

Biological Analogy: Evolution [8].

- **Population:** A set of solutions (Chromosomes).
- **Crossover:** Combine parents to create offspring (e.g., Order Crossover).
- **Mutation:** Randomly tweak a child to maintain diversity.
- **Selection:** Survival of the fittest.

VRP Specialization: Most successful VRP GAs use a **"Giant Tour"** representation (a single sequence of all customers) and a **Split Algorithm** to optimally break the sequence into routes.

Ant Colony Optimization (ACO)

Biological Analogy: Foraging ants [10].

- **Pheromones:** Artificial agents (ants) construct routes probabilistically based on "pheromone trails" τ_{ij} on edges.
- **Update:** Good routes receive more pheromone; unused edges evaporate.
- **Result:** Over time, the colony converges on the optimal path.

Large Neighborhood Search (LNS)

Mechanism: Ruin and Recreate [9].

- **Ruin:** Intentionally destroy part of the solution (e.g., remove 20% of customers randomly or spatially).
- **Recreate:** Re-insert the removed customers using a greedy heuristic or regret heuristic.
- **ALNS (Adaptive LNS):** Automatically selects which "Ruin" and "Recreate" operators work best for the current instance.

Exact Algorithm Implementation

Two independent Python implementations were developed to solve the CVRP exactly, both tested on standard `.vrp` benchmark instances.

Aspect	**Custom B&C**	**OR-Tools**
LP / MIP engine	`scipy` HiGHS	OR-Tools CP engine
Cuts added	Capacity + Comb	Internal (hidden)
First solution	Nearest-neighbour + 2-opt	Multi-strategy
Improvement	B&B with LIFO stack	GLS / Tabu / SA
Optimality proof	Yes (within node budget)	Yes (time limit)
Scalability	$n \lesssim 50$	$n \lesssim 200$
Dependencies	`numpy, scipy`	`ortools`

Both solvers accept standard TSPLIB `.vrp` files and are executed via the shell scripts `run_bc.sh` and `run_all.sh` for batch benchmarking.

Results: Custom B&C vs. OR-Tools (1/2)

Experimental Setup

Tested on **27 Augerat Set-A benchmarks** ($n = 31$–79 customers). **Time limits:** OR-Tools = 360 s, Custom B&C = 600 s. Known optimal distances used as ground truth.

Instance	n	k	Optimal	B&C	B&C Gap (%)	OR-Tools	ORT Gap (%)
A-n32-k5	31	5	784	926	18.11	784	0.00
A-n33-k5	32	5	661	661	0.00	661	0.00
A-n33-k6	32	6	742	836	12.67	742	0.00
A-n34-k5	33	5	778	805	3.47	778	0.00
A-n36-k5	35	5	799	1031	29.04	799	0.00
A-n37-k5	36	5	669	756	13.00	669	0.00
A-n37-k6	36	6	949	1271	33.93	949	0.00
A-n38-k5	37	5	730	986	35.07	730	0.00
A-n39-k5	38	5	822	1028	25.06	825	0.36
A-n39-k6	38	6	831	957	15.16	833	0.24
A-n44-k6	43	6	937	1343	43.33	939	0.21
A-n45-k6	44	6	944	1422	50.64	951	0.74
A-n45-k7	44	7	1146	1383	20.68	1146	0.00

Results: Custom B&C vs. OR-Tools (2/2)

Instance	n	k	Optimal	B&C	B&C Gap (%)	OR-Tools	ORT Gap (%)
A-n48-k7	47	7	1073	1453	35.41	1073	0.00
A-n53-k7	52	7	1010	1385	37.13	1017	0.69
A-n54-k7	53	7	1167	1386	18.77	1170	0.26
A-n55-k9	54	9	1073	1472	37.19	1074	0.09
A-n60-k9	59	9	1354	1775	31.09	1354	0.00
A-n62-k8	61	8	1288	1713	33.00	1306	1.40
A-n63-k10	62	10	1314	1860	41.55	1321	0.53
A-n63-k9	62	9	1616	2139	32.36	1641	1.55
A-n64-k9	63	9	1401	1903	35.83	1419	1.28
A-n65-k9	64	9	1174	1694	44.29	1184	0.85
A-n69-k9	68	9	1159	1477	27.44	1170	0.95
A-n80-k10	79	10	1763	2290	29.89	1791	1.59
Average					**28.08**		**1.06**

Custom B&C (limit: 600 s)

- Avg. gap: **28.08%**
- Gap range: 0% – 50.64%

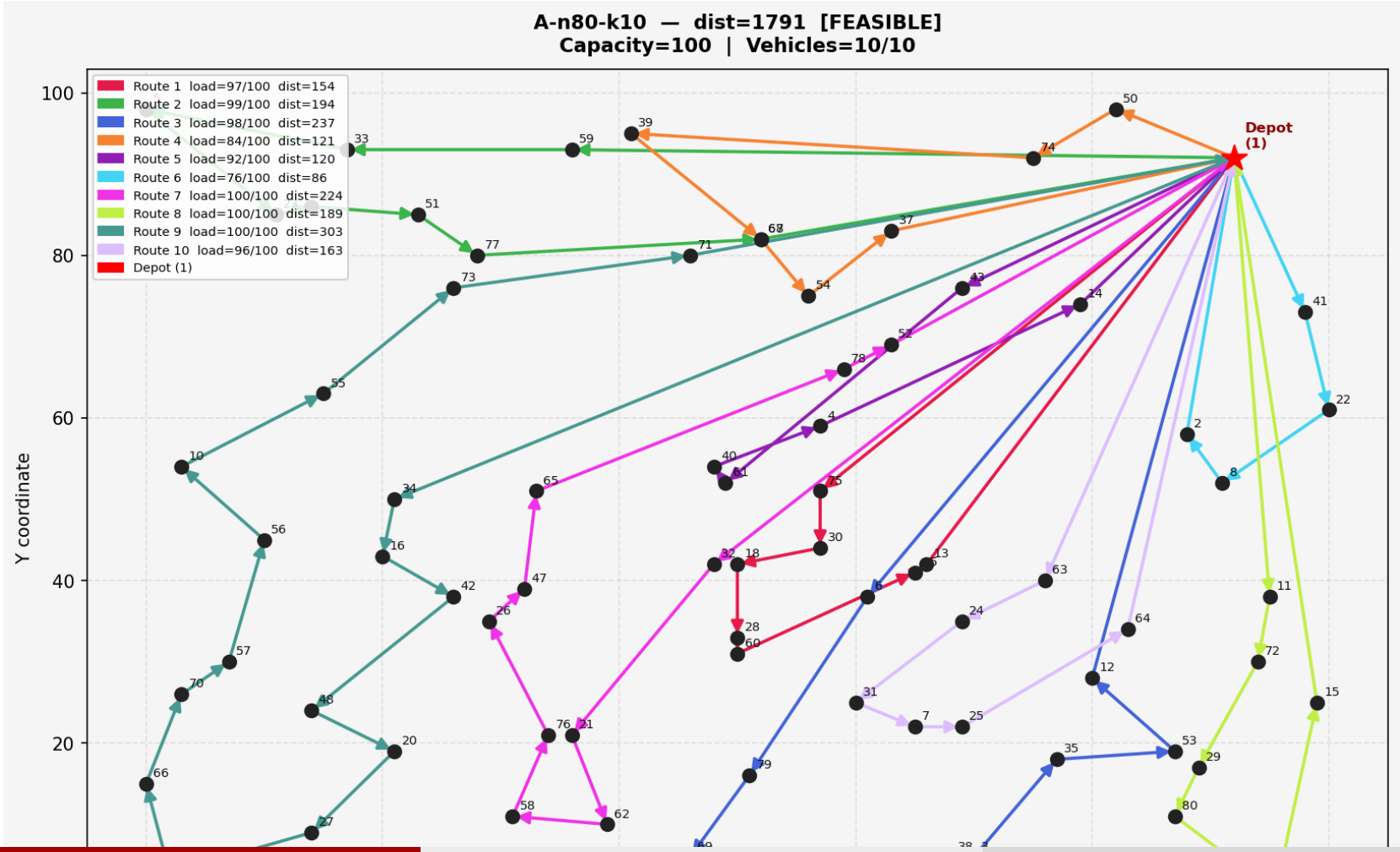
OR-Tools (limit: 360 s)

- Avg. gap: **1.06%**
- Gap range: 0% – 1.59%

OR-Tools Solution — A-n80-k10

Instance: A-n80-k10 $n = 79$, $k = 10$, Optimal $= 1763$

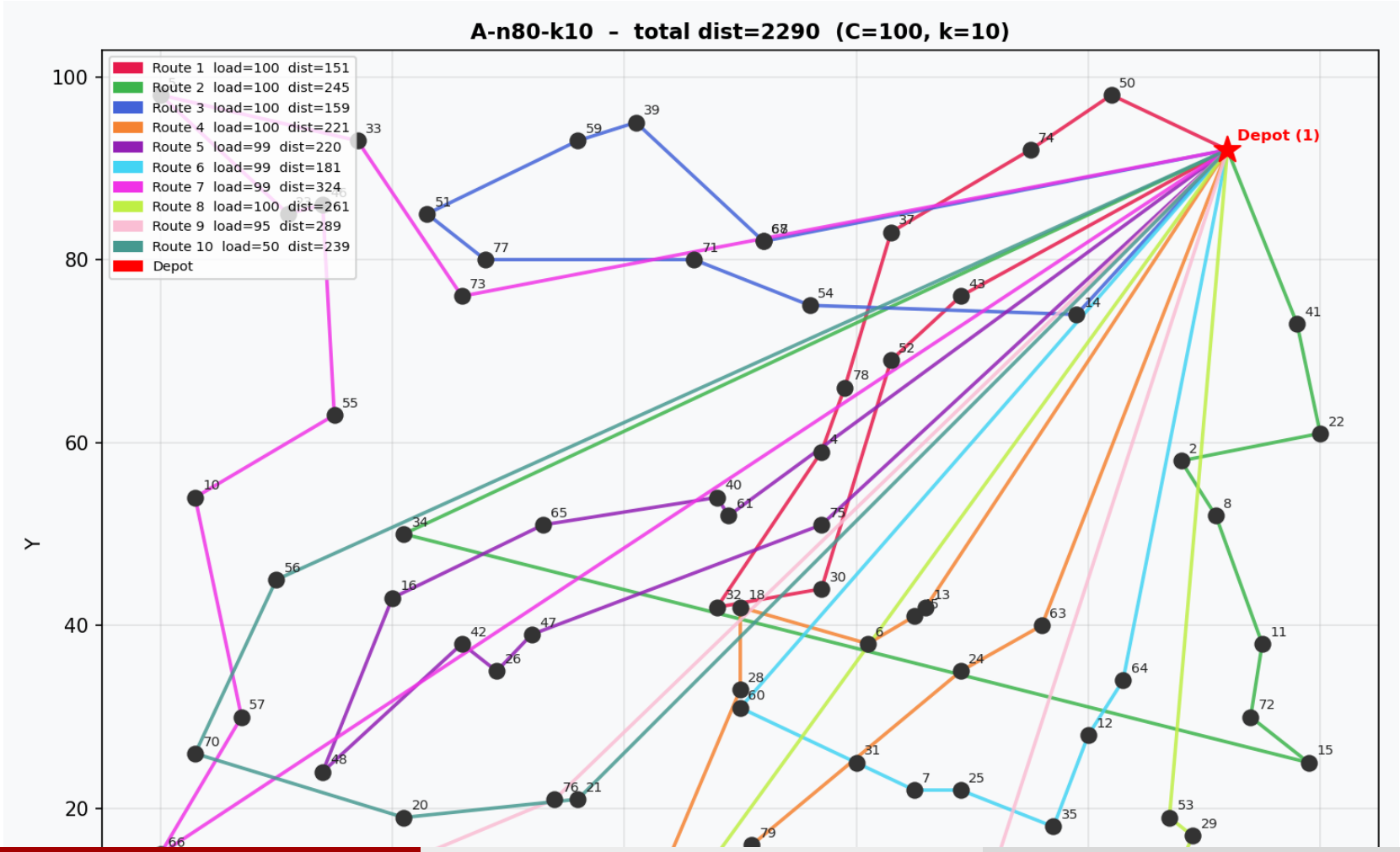
OR-Tools result: **1791** Gap: **1.59%** Time limit: 360 s



Custom B&C Solution — A-n80-k10

Instance: A-n80-k10 $n = 79$, $k = 10$, Optimal $= 1763$

B&C result: **2290** Gap: **29.89%** Time limit: 600 s



References I

[1] Philippe Augerat et al. *Computational results with a branch and cut code for the capacitated vehicle routing problem*. Tech. rep. RR-949-M. Université Joseph-Fourier, Grenoble, France, 1995.

[2] Gilbert Laporte and Yves Nobert. "A branch and bound algorithm for the capacitated vehicle routing problem". In: *OR Spektrum* 5 (1983), pp. 77–85. DOI: 10.1007/BF01720015.

[3] Ricardo Fukasawa et al. "Robust branch-and-cut-and-price for the capacitated vehicle routing problem". In: *Mathematical Programming* 106.3 (2006), pp. 491–511. DOI: 10.1007/s10107-005-0644-x.

[4] Nicos Christofides and Samuel Eilon. "Set M and CMT instances". In: *Summary of the VRPNC Benchmark Instances* (1979).

References II

[5] Amina Shour, Kassem Danash, and Abbas Tarhini. "Modified Clarke Wright Algorithms for Solving the Realistic Vehicle Routing Problem". In: *2015 Third International Conference on Technological Advances in Electrical, Electronics and Computer Engineering (TAEECE)*. 2015, pp. 89–93. DOI: 10.1109/TAEECE.2015.7113606.

[6] Neha Sehta and Urjita Thakar. "Sweep Nearest Algorithm for Capacitated Vehicle Routing Problem". In: *2021 IEEE 18th India Council International Conference (INDICON)*. 2021, pp. 1–6. DOI: 10.1109/INDICON52576.2021.9691603.

[7] Gulay Barbarosoglu and Demet Ozgur. "A tabu search algorithm for the vehicle routing problem". In: *Computers & Operations Research* 26.3 (1999), pp. 255–270. DOI: 10.1016/S0305-0548(98)00047-1.

References III

[8] Ali Najm Jasim and Lamia Chaari Fourati. "Guided Genetic Algorithm for Solving Capacitated Vehicle Routing Problem With Unmanned-Aerial-Vehicles". In: *IEEE Access* 12 (2024), pp. 106333–106358. DOI: 10.1109/ACCESS.2024.3438079.

[9] Wenjuan Gu et al. "Adaptive large neighborhood search for the commodity constrained split delivery VRP". In: *Computers & Operations Research* 112 (2019), p. 104761. DOI: 10.1016/j.cor.2019.07.019.

[10] Petr Stodola et al. "Using the Ant Colony Optimization Algorithm for the Capacitated Vehicle Routing Problem". In: *Proceedings of the 16th International Conference on Mechatronics – Mechatronika 2014*. 2014, pp. 503–510. DOI: 10.1109/MECHATRONIKA.2014.7018311.

Thank You

Thank You!

All source code, benchmark datasets, solution outputs, and route plots are available at:

`https://github.com/TH-014/CVRP.git`

The repository includes implementations of the custom Branch-and-Cut solver, the OR-Tools-based solver, Sweep algorithm and Genetic algorithm based solver and their modified versions, shell scripts for batch benchmarking, and all generated plots for the Augerat Set-A instances.